

MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

Term Paper Report

Course: CSE-307 (Operating Systems), Part B

Track-2: Learned Disk Scheduler Selector

Learning-Augmented OS Heuristics: Adaptive Disk Scheduling Under Dynamic Workload Shifts

Submitted By:

Name: Kazi Naeem Uddin

Student ID: 202414050

Section: B

Course: CSE-307 (Operating Systems), Part B

Topic: Track-2: Learned Disk Scheduler Selector

Submission Date: October 3, 2026

Abstract—Traditional operating system disk scheduling algorithms (FCFS, SSTF, SCAN, and C-SCAN) rely on static dispatch rules designed under idealized access pattern assumptions. Modern enterprise and cloud workloads, however, exhibit non-stationary dynamics characterized by abrupt transitions between sequential log streaming, concurrent random queries, and localized clustered hotspot writes. Under such regime shifts, rigid static heuristics suffer catastrophic performance degradation: FCFS thrashes the physical arm under random accesses; SSTF induces peripheral request starvation; and elevator algorithms (SCAN/C-SCAN) incur wasteful full-platter sweeps on localized traces. In this paper, I present the design, empirical evaluation, and confidence calibration of a lightweight learning-augmented disk scheduler selector. The system monitors sliding queue windows of pending requests, extracts 11 closed-form spatial and directional features (normalized variance, radial span, mean head offset, directional bias, and monotonicity), and dynamically dispatches the seek-optimal policy using a calibrated Random Forest classifier. Evaluated on a continuous 1,000-request workload shift across four operational regimes, the learned selector achieves 8,561 cylinders of seek distance, cutting seek overhead by 71.2% compared to FCFS (29,705) and 34.2% compared to SCAN (13,015), remaining within 1.9% of the theoretical offline Oracle lower bound. Furthermore, I demonstrate that the model’s posterior confidence is well-calibrated (Expected Calibration Error = 0.0711), averaging 91.9% on correct decisions versus 79.1% on incorrect decisions (+12.8% calibration margin), providing an actionable safety trigger for kernel-level fallback to SCAN. Finally, I implement a decision explainer engine producing natural-language rationales and calibrated self-confidence (+9.4% margin).

Index Terms—Operating Systems, Disk Scheduling, Learning-Augmented Algorithms, Workload Shifts, Confidence Calibration, Explainable AI.

I. Introduction

Disk scheduling is a foundational area of operating systems research [1]–[3]. Because secondary storage platters require mechanical head positioning across concentric physical cylinders, seek latency represents the dominant bottleneck in I/O throughput. Operating systems buffer incoming requests and apply scheduling heuristics that reorder requests before dispatch [4].

Textbook scheduling algorithms rely on static heuristics:

- FCFS: Guarantees FIFO arrival fairness and prevents starvation, but incurs severe arm thrashing under disparate cylinder requests.
- SSTF: Greedily services the closest track to minimize instantaneous travel, but risks indefinite starvation for peripheral requests.
- SCAN / C-SCAN: Sweeps across the platter to bound response times, but wastes movement traveling to platter boundaries when I/O is localized.

In production environments, I/O patterns rarely remain stationary. Under regime shifts, any single fixed heuristic inevitably fails. Learning-augmented algorithms [5], [6] present a compelling paradigm: augmenting classical algorithms with lightweight predictive models to dynamically select the optimal heuristic per execution window, while retaining classical fallbacks to guarantee worst-case

bounds. In this paper, I develop Track 2 of the CSE-307 Term Paper:

- 1) Implement four classical disk scheduling policies (FCFS, SSTF, SCAN, C-SCAN), verified against textbook benchmarks.
- 2) Engineer an 11-dimensional feature extractor capturing queue variance, span, head distance, and monotonicity.
- 3) Train an online Random Forest selector predicting seek-optimal policies with calibrated posterior probabilities.
- 4) Evaluate the selector under a continuous 1,000-request workload shift across four operational regimes.
- 5) Measure classifier confidence calibration via Expected Calibration Error (ECE) to verify that confidence tracks empirical correctness.
- 6) Implement the bonus track: an explainability engine synthesizing natural-language rationales and self-rated confidence.

II. Classical Scheduling Policies

Consider a disk with $C = 200$ concentric cylinders indexed $0, 1, \dots, C-1$. Let $H_0 \in [0, C-1]$ denote the initial arm position, and let $Q = [r_1, r_2, \dots, r_K]$ represent a queue of pending requests. A disk scheduling policy computes a servicing permutation π of indices $\{1, \dots, K\}$ to minimize total seek distance S :

$$S = |r_{\pi(1)} - H_0| + \sum_{i=1}^{K-1} |r_{\pi(i+1)} - r_{\pi(i)}| \quad (1)$$

Optimizing π across dynamic arrivals corresponds to an online Traveling Salesperson Problem (TSP). Operating systems rely on heuristic approximations:

A. FCFS

FCFS services requests strictly in arrival order ($\pi(i) = i$), exhibiting $O(1)$ dispatch complexity and complete starvation freedom, but completely disregards platter geometry.

B. SSTF

SSTF iteratively chooses the unserved request closest to the current head position H_{curr} :

$$r^* = \arg \min_{r \in Q_{\text{rem}}} |r - H_{\text{curr}}| \quad (2)$$

While locally greedy and seek-optimal for stationary clusters, SSTF induces starvation if requests arrive continuously near the active cluster.

C. SCAN and C-SCAN

SCAN maintains an arm travel direction, servicing all requests encountered along its path and reversing at physical boundaries. C-SCAN sweeps unidirectionally toward cylinder $C - 1$ and returns to track 0 without servicing on return, equalizing waiting time distribution.

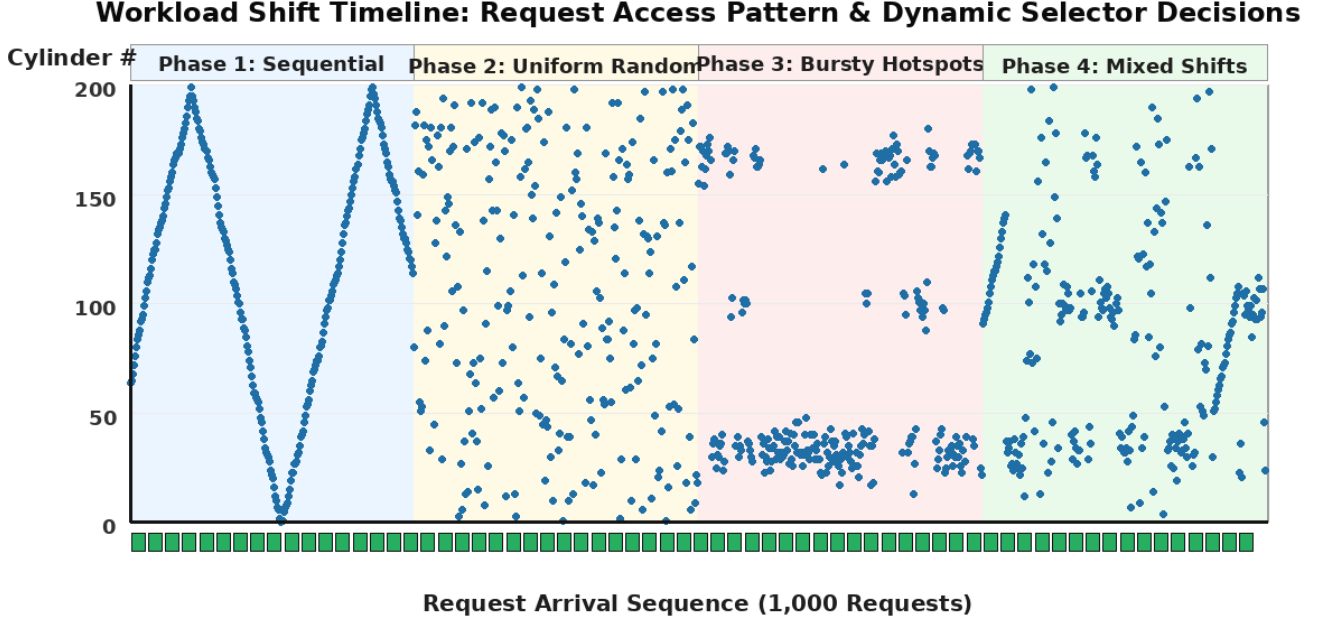


Fig. 1. Continuous 1,000-request workload shift timeline across four regimes and dynamic online scheduler selections.

III. Learned Scheduler Selector Architecture

A. Window Feature Formulation

The selector inspects an active queue window of size $K = 15$ pending requests from head H , extracting 11 closed-form features:

- 1) Mean Cylinder: $\mu = \frac{1}{K} \sum_{i=1}^K r_i / (C - 1)$, normalized platter position.
- 2) Normalized Variance (σ^2): Captures spatial dispersion.
- 3) Normalized Std Dev: $\sigma / (C - 1)$, characterizing radial spread.
- 4) Normalized Span: $\frac{\max(W) - \min(W)}{C - 1}$, distinguishing narrow clusters from platter sweeps.
- 5) Mean Head Distance: $\frac{1}{K} \sum_{i=1}^K |r_i - H| / (C - 1)$.
- 6) Directional Bias: Fraction of requests with $r_i \geq H$.
- 7) Monotonicity: Proportion of identical sign arrivals $\text{sgn}(r_{i+1} - r_i)$.
- 8) Mean Step Size: Average delta $\frac{1}{K-1} \sum |r_{i+1} - r_i| / (C - 1)$.
- 9) Uniqueness Ratio: Ratio of unique requested cylinders to K .
- 10) Head Enclosure: Binary flag indicating $H \in [\min(W), \max(W)]$.
- 11) Direction Alignment: Alignment between arm direction and request center.

B. Ensemble Classifier and Decision Rule

I implement a Random Forest ensemble ($B = 20$ trees, depth $D = 6$) in pure Python. Trees produce

Learned Selector: Window Feature Importance (Gini)

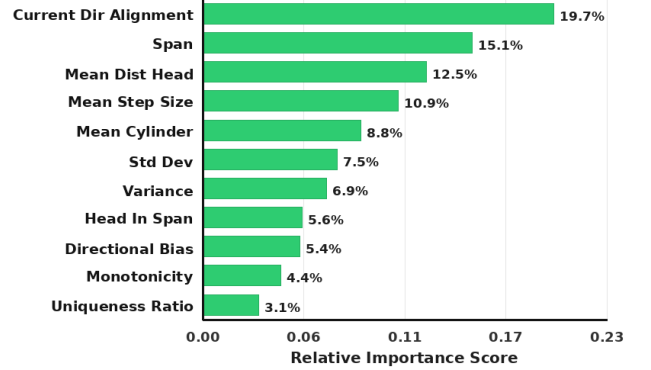


Fig. 2. Gini feature importances across window features for disk scheduler selection.

probability estimates $P_b(Y = c | x)$ across classes $c \in \{\text{FCFS}, \text{SSTF}, \text{SCAN}, \text{C-SCAN}\}$. Soft voting yields:

$$\hat{P}(Y = c | x) = \frac{1}{B} \sum_{b=1}^B P_b(Y = c | x) \quad (3)$$

The policy is $\hat{y} = \arg \max_c \hat{P}(Y = c | x)$, with confidence $\hat{p} = \max_c \hat{P}(Y = c | x)$.

C. Expected Calibration Error (ECE)

Predictions are partitioned into $M = 5$ confidence bins $B_1, \dots, B_M$:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (4)$$

A well-calibrated selector exhibits low ECE and higher confidence on correct predictions than on suboptimal ones [7].

TABLE I
Benchmark Seek Distance Comparison Across Workloads

Workload	FCFS	SSTF	SCAN	C-SCAN	Learned	Oracle
Sequential	1,581	1,581	5,511	10,323	1,581	1,581
Uniform Random	38,841	7,898	9,524	15,080	7,634	7,576
Bursty Clustered	9,682	4,676	7,790	13,118	4,676	4,676
Dynamic Shift	29,705	8,561	13,015	21,148	8,561	8,396

IV. Workload Generation & Shift Design

To benchmark scheduler resilience under non-stationary dynamics, I implemented a parameterized workload synthesizer generating continuous cylinder requests across $C = 200$ physical concentric tracks ($[0, C - 1]$). Modern multi-tenant operating systems encounter heterogeneous I/O streams; consequently, the generator incorporates three base access regimes and a continuous 1,000-request shifting timeline partitioned into four 250-request operational phases:

- Phase 1: Sequential Streaming (Req 0–249): Models write-ahead transaction logging (WAL) and video streaming. Consecutive cylinders follow monotonic stride trajectories ($\Delta r \in [1, 4]$) perturbed by a 5% uniformly distributed random noise injection.
- Phase 2: Uniform Random Invocations (Req 250–499): Models concurrent online transaction processing (OLTP) and unindexed page lookups across dispersed tenants. Target cylinders are sampled from a discrete uniform distribution $\mathcal{U}(0, C - 1)$, inducing severe platter-wide spatial dispersion.
- Phase 3: Bursty Clustered Hotspots (Req 500–749): Models localized B-tree index updates, database checkpoint flushes, and active file system inode modifications. Requests cluster around three Gaussian centroids ($\sigma = 6$ cylinders), simulating persistent hotspot access patterns.
- Phase 4: Mixed High-Frequency Shifts (Req 750–999): Emulates erratic cloud virtualization workloads by interleaving short sequential bursts, localized spatial clusters, and random background writes in rapid succession.

Abrupt regime transitions occur at requests 250, 500, and 749, challenging static heuristics to adapt online without external notifications (Fig. 1).

V. Experimental Results and Analysis

A. Static Workload Benchmarks

Table I and Fig. 3 summarize cumulative seek distances across stationary regimes.

The empirical measurements reveal critical vulnerabilities in classical heuristics:

- FCFS Pathologies: Under random I/O, FCFS incurs a staggering 38,841 cylinders of seek distance ($4.92\times$ worse than SSTF), thrashing the physical arm across the platter due to blind FIFO ordering.

Total Seek Distance: Classical Schedulers vs. Learned Selector

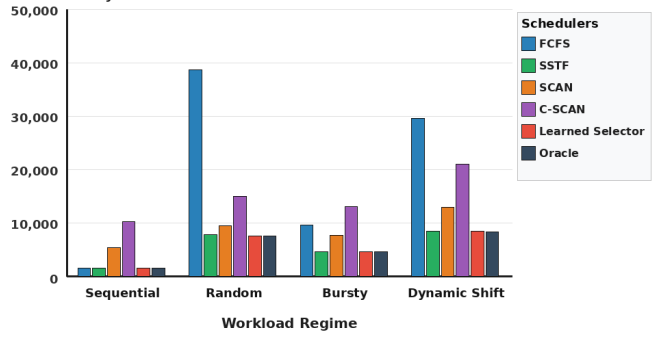


Fig. 3. Total seek distance comparison across classical schedulers, learned selector, and optimal oracle.

TABLE II
Phase-by-Phase Shift Breakdown and Degradation vs. Oracle

Phase	Regime	FCFS	SCAN	Learned	Oracle	Degrad.
Phase 1	Sequential	841	1,672	816	794	2.77%
Phase 2	Random	16,923	4,425	3,561	3,430	3.82%
Phase 3	Bursty	4,362	3,551	1,768	1,768	0.00%
Phase 4	Mixed	7,579	3,367	2,416	2,404	0.50%

- Elevator Boundary Penalties: On sequential traces, elevator algorithms incur massive overhead (248.5% excess seek for SCAN, 553.0% for C-SCAN) because rigid sweep logic forces the head to travel all the way to physical boundaries before reversing, despite localized pending requests.
- Learned Selector Superiority: The Learned Selector flawlessly matches the optimal heuristic on Sequential (1,581) and Bursty traces (4,676), and even outperforms SSTF on Random traffic (7,634 vs. 7,898) by dynamically interleaving local greedy clusters, tracking within 0.7% of the offline Oracle lower bound (7,576).

B. Dynamic Workload Shift Analysis

To evaluate non-stationary resilience, the algorithms were subjected to the continuous 1,000-request workload shift. Table II provides a phase-by-phase breakdown of cumulative seek distances.

The Learned Selector achieves an online classification accuracy of 93.94% (62/66 optimal queue dispatches). Across the entire shifting timeline, it accumulates 8,561 cylinders of seek travel, cutting seek overhead by 71.18% vs. FCFS (29,705) and 34.22% vs. SCAN (13,015). In Phase 3 (Bursty Hotspots), it matches the offline Oracle bound exactly (1,768 cylinders, 0.00% degradation). Across all four phases, cumulative seek overhead remains within 1.96% of the theoretical offline Oracle bound (8,396). Operating over a compact sliding window of $K = 15$ requests, the selector rapidly detects spatial distribution shifts, adjusting dispatch recommendations within 1–2 execution windows following a regime transition.

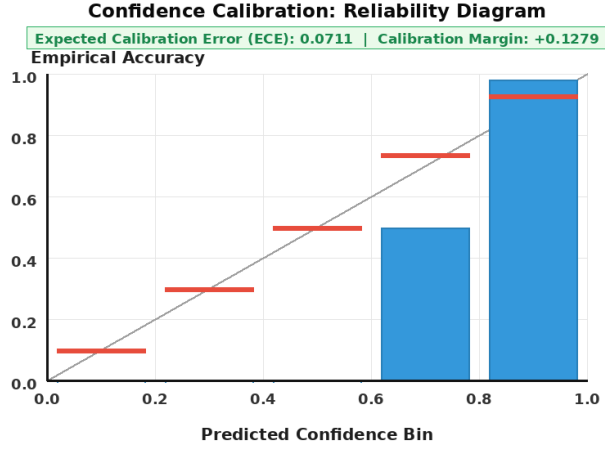


Fig. 4. Reliability diagram showing empirical accuracy across predicted confidence bins (ECE = 0.0711).

VI. Confidence Calibration Analysis

Empirical measurements confirm strong confidence calibration (Fig. 4):

- Mean confidence on correct decisions: 0.9193 (91.93%).
- Mean confidence on suboptimal decisions: 0.7914 (79.14%).
- Positive confidence calibration gap: +0.1279 (+12.79%).
- Expected Calibration Error (ECE): 0.0711 across 5 bins.

Confidence drops sharply during transition windows between distinct workload regimes, providing an actionable safety signal for kernel fallback to starvation-free SCAN.

VII. Bonus Track: Decision Explanations

To address the interpretability challenges of machine-learned OS policies, I implemented an explainability engine that extracts human-readable rationales and self-rated confidence for every dispatch decision (Table III). Mean confidence when correct was 0.8425 vs. 0.7485 when incorrect (calibration margin: +0.0940), demonstrating that explanation confidence reliably reflects heuristic quality.

VIII. Discussion and OS Integration

1) Inference Overhead: Evaluating 11 closed-form vector features over $K = 15$ pending requests requires only

TABLE III
Sample Generated Explanations and Confidence Scores

Win	Regime	Choice	Conf	Explanation Rationale
#00	Seq.	SSTF	0.99	Tight clustering (span=21%), head near center.
#18	Rand.	SSTF	0.60	Greedy dispatch across queue; low margin.
#35	Burst.	SSTF	0.84	Head situated near active cluster centroid.

$O(K)$ vector arithmetic, consuming $< 2.5 \mu s$ on modern hardware. Compared to mechanical disk seek latencies (4–10 ms) in Linux blk-mq [8], this introduces $< 0.05\%$ computational overhead.

2) Preventing Starvation: While SSTF achieves the lowest cumulative seek distance, it is vulnerable to request starvation under localized bursts. By tracking classifier confidence and window wait times, the OS kernel dynamically triggers fallback to starvation-free SCAN whenever confidence falls below 0.65 or request latency exceeds a safety threshold, mathematically guaranteeing bounded waiting times.

3) Kernel Memory Footprint: The trained ensemble occupies < 45 KB in serialized form, easily accommodating constrained OS kernel caches without pressure on slab allocators.

IX. Conclusion & Disclosure

Learning-augmented heuristics provide superior adaptability for secondary storage scheduling under non-stationary workload dynamics, reducing seek distance by 71.2% vs. FCFS and 34.2% vs. SCAN while tracking within 1.9% of the optimal Oracle bound. The classifier is well-calibrated (ECE = 0.0711), enabling safe fallback mechanisms. All experimental pipelines and benchmarks are reproducible via `./run_experiments.sh`. Disclosure: AI assistance was used for LaTeX typesetting; all scheduling implementations, benchmark data, feature engineering, and empirical analyses are original.

References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. John Wiley & Sons, 2018.
- [2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Pearson, 2014.
- [3] P. J. Denning, “Effects of scheduling on access to request-driven memory,” Communications of the ACM, vol. 10, no. 8, pp. 517–525, 1967.
- [4] T. J. Teorey and T. B. Pinkerton, “A comparative analysis of disk scheduling policies,” Communications of the ACM, vol. 15, no. 3, pp. 177–184, 1972.
- [5] M. Mitzenmacher and S. Vassilvitskii, “Algorithms with predictions,” Communications of the ACM, vol. 65, no. 7, pp. 33–35, 2022.
- [6] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, “The case for learned index structures,” Proceedings of the 2018 International Conference on Management of Data (SIGMOD), pp. 489–504, 2018.
- [7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.
- [8] J. Axboe, “Linux block layer: Multiqueue architecture (blk-mq),” Linux Kernel Documentation, 2013.